



ITM SKILLS
UNIVERSITY

PROJECT NUMBER

140

DRAGGABLE AUDIO-PANNED MULTI-CHANNEL SPATIAL AUDIO SYSTEM

Academic Project Documentation

Submitted By Om Kumar Maurya

Roll Number 150096725208

Program Bachelor of TechnologyComputer Science
and Engineering

Year First Year

Cohort Jeff Bezos

Submission Year 2026

GitHub Repository:

<https://github.com/omkumarmaurya99-source/Draggable-Audio-Panned-Multi-Channel-Playground.git>

Live Demo:

<https://draggable-audio-panned-multi-CHANNE-five.vercel.app/>

Table of Contents

1. Executive Summary

2. Introduction

3. Problem Statement

4. Project Objectives

5. Project Overview

6. Technology Stack

7. System Architecture

7.1 Application Flow

7.2 Audio Processing Architecture

7.3 State Management Flow

8. Component Architecture

9. Core Features

10. Spatial Audio Mathematics

11. Implementation Details

12. User Workflow

13. Screenshots

14. Testing & Verification

15. Challenges Faced

16. Learning Outcomes

17. Future Improvements

18. Conclusion

19. References

1. EXECUTIVE SUMMARY

The Draggable Audio-Panned Multi-Channel Spatial Audio System is an interactive web application that enables users to visualize and manipulate spatial audio in a two-dimensional environment. Built using React.js and the Web Audio API, the application provides an educational platform for understanding fundamental concepts of sound propagation, stereo panning, and distance-based volume attenuation.

The system features a virtual sound stage where users can drag audio sources around a fixed listener position. As sources are repositioned, the application calculates real-time Euclidean distances to dynamically adjust stereo pan values and volume attenuation, thereby simulating how sound behaves in physical space. The project incorporates a channel mixer for per-source volume control, a telemetry dashboard for monitoring audio parameters, and a persistence layer utilizing browser local storage and JSON export/import functionality.

This documentation provides a comprehensive overview of the project's architecture, mathematical foundations, component design, implementation details, and the learning outcomes achieved through its development.

2. INTRODUCTION

Spatial audio refers to the technique of processing sound so that it appears to originate from specific locations in a three-dimensional or two-dimensional space around the listener. Unlike traditional stereo audio, which simply distributes sound between left and right channels, spatial audio creates the illusion of directionality and depth by manipulating parameters such as volume, panning, and timing.

Modern web browsers have made it possible to perform sophisticated audio processing directly on the client side through the Web Audio API. This browser-native API provides a powerful and efficient system for controlling audio on the web, offering nodes for gain control, stereo panning, oscillator generation, and waveform analysis.

This project combines the capabilities of the Web Audio API with the component-based architecture of React.js to create an intuitive, real-time spatial audio simulator. It serves both as an educational tool for learning audio processing concepts and as a practical demonstration of modern frontend engineering techniques.

3. PROBLEM STATEMENT

Traditional stereo audio systems present sound through only two fixed channels—left and right—without regard for the spatial positioning of individual sound sources. This approach is inherently limited because:

- No positional context: Listeners cannot perceive the direction or distance of a sound source relative to their position.

- Static audio presentation: Conventional audio mixing does not dynamically respond to changes in source positioning.
- Lack of educational tools: Existing spatial audio implementations are often embedded in complex game engines or professional audio software, making them inaccessible to students and beginners.
- No browser-based interactive simulators: There is a scarcity of lightweight, web-based tools that allow users to visually explore spatial audio principles without installing specialized software.

This project addresses these limitations by providing a browser-based spatial audio simulator that visually demonstrates how distance, angle, and position affect audio perception. The system calculates real-time stereo panning and distance-based attenuation, enabling users to intuitively understand spatial audio concepts through direct interaction.

Expected Outcomes

- A functional 2D spatial audio playground accessible via any modern web browser.
- Real-time audio parameter updates based on source-listener geometry.
- An educational platform suitable for demonstrating audio processing fundamentals.
- A deployable application with state persistence and configuration management.

4. PROJECT OBJECTIVES

The primary objectives of this project are as follows:

1. Develop an interactive 2D sound stage where users can place and drag audio source nodes around a fixed listener position.
2. Implement real-time spatial audio processing using the Web Audio API, including stereo panning and distance-based volume attenuation.
3. Build a multi-channel mixer providing independent volume, pan, play/pause, and mute controls for each audio source.
4. Create a telemetry dashboard displaying real-time audio parameters including distance, pan value, attenuation factor, and collision status.
5. Implement state persistence through browser local storage and JSON-based export/import functionality.
6. Support both oscillator-generated audio and uploaded audio files as sound sources.
7. Deploy the application as a publicly accessible web application via Vercel.

5. PROJECT OVERVIEW

The Draggable Audio-Panned Multi-Channel Spatial Audio System is a single-page application built with React.js and bundled using Vite. The application presents users with a 2D grid-based sound stage containing a fixed central listener node surrounded by movable audio source nodes.

Purpose

The application serves as an educational and experimental platform for understanding how spatial positioning of sound sources affects audio perception. By providing visual feedback through draggable nodes and real-time telemetry, the system bridges the gap between abstract audio theory and practical interaction.

Core Workflow

Upon launching the application, users are presented with the sound stage pre-loaded with default oscillator sources. Users can add new sources (either oscillators or uploaded audio files), drag them around the stage, and observe how the audio output changes in real time. The mixer panel provides granular control over each channel, while the telemetry dashboard displays the underlying mathematical parameters driving the audio processing.

Educational Value

This system demonstrates several core computer science and audio engineering concepts: coordinate geometry (Euclidean distance), signal processing (gain control, stereo panning), event-driven programming (drag-and-drop), component architecture (React), and browser API integration (Web Audio API). It provides a hands-on learning experience that is significantly more engaging than textbook-only approaches.

6. TECHNOLOGY STACK

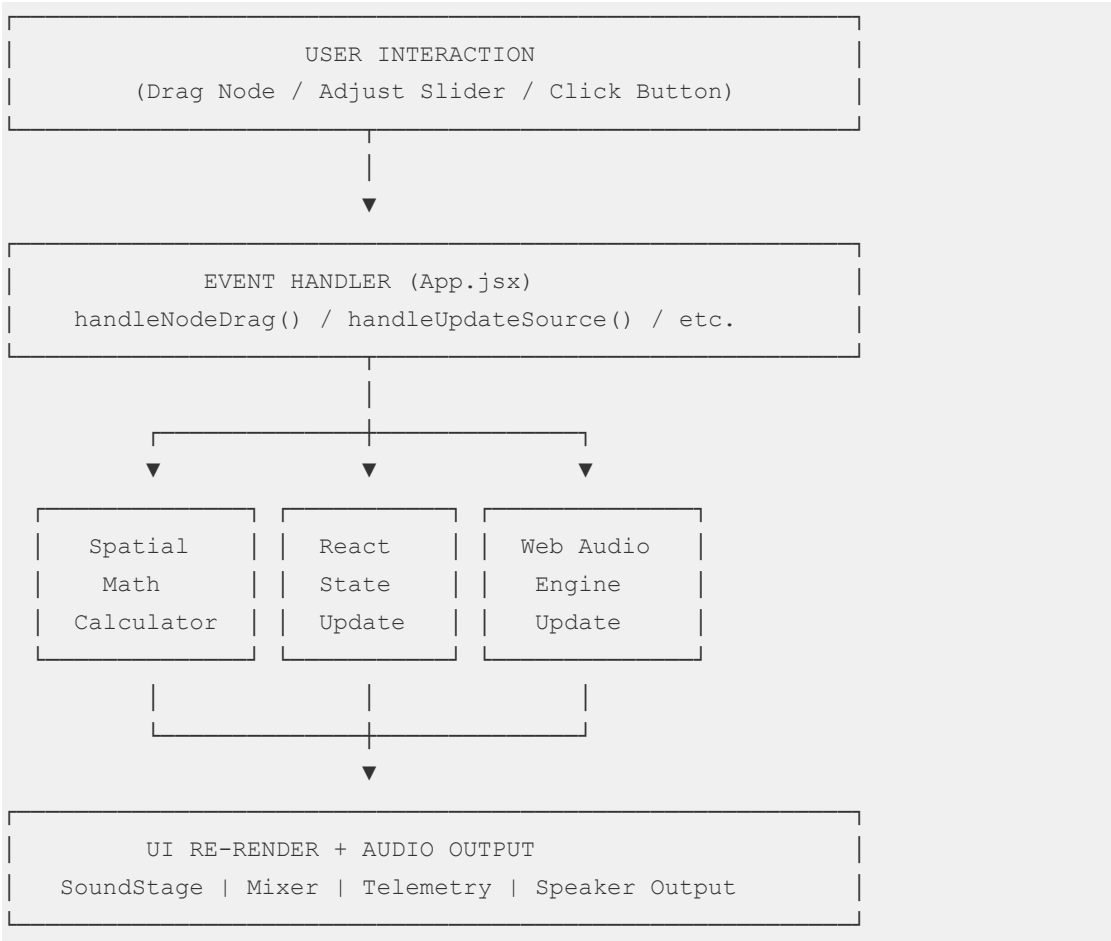
Technology	Version	Purpose & Justification
React.js	18.x	Component-based UI library for building interactive user interfaces. React's declarative paradigm and virtual DOM enable efficient real-time updates when audio parameters change during drag operations.
Vite	6.x	Next-generation frontend build tool. Vite provides near-instant Hot Module Replacement (HMR) during development and optimized bundling for production.
JavaScript (ES6+)	ES2020+	Core programming language used for all application logic, mathematical calculations, event handling, and Web Audio API integration.
Web Audio API	Browser Native	Low-latency, high-fidelity audio processing API. Provides AudioContext, OscillatorNode, GainNode, StereoPannerNode, and AnalyserNode for real-time audio synthesis, routing, and analysis.

Technology	Version	Purpose & Justification
HTML5	5	Semantic markup language providing the structural foundation. HTML5 Canvas is used for drawing distance vectors and collision bridge overlays on the sound stage.
CSS3	3	Styling language used for the dark-themed studio interface, glassmorphism panels, neon accent colors, animations, and responsive grid layouts.
Browser LocalStorage	Web Standard	Client-side key-value storage mechanism used to persist audio source positions, settings, and configurations across browser sessions without requiring a backend server.

7. SYSTEM ARCHITECTURE

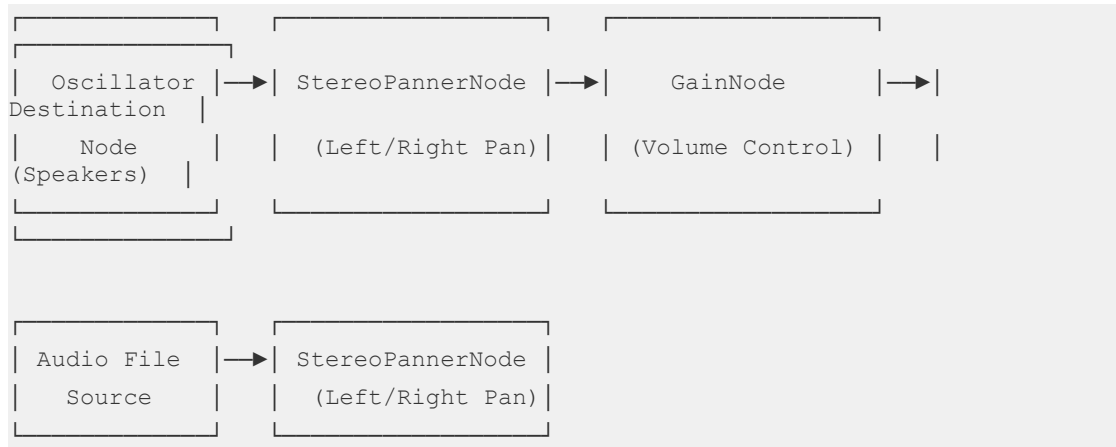
7.1 Application Flow

The application follows a unidirectional data flow pattern consistent with React best practices. All state is managed centrally in the App component and passed down to child components via props. User interactions trigger callbacks that update the central state, which in turn triggers re-renders of affected components and synchronous updates to Web Audio API nodes.



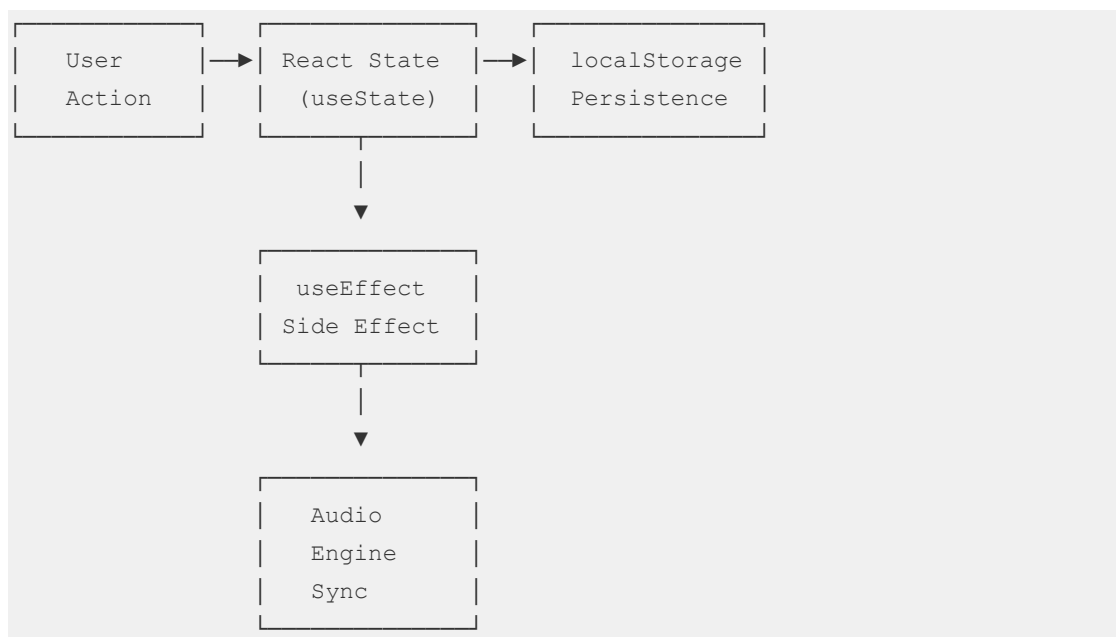
7.2 Audio Processing Architecture

The audio processing pipeline is constructed using the Web Audio API's node-graph architecture. Each audio source is connected through a chain of processing nodes before reaching the final destination (speakers).



7.3 State Management Flow

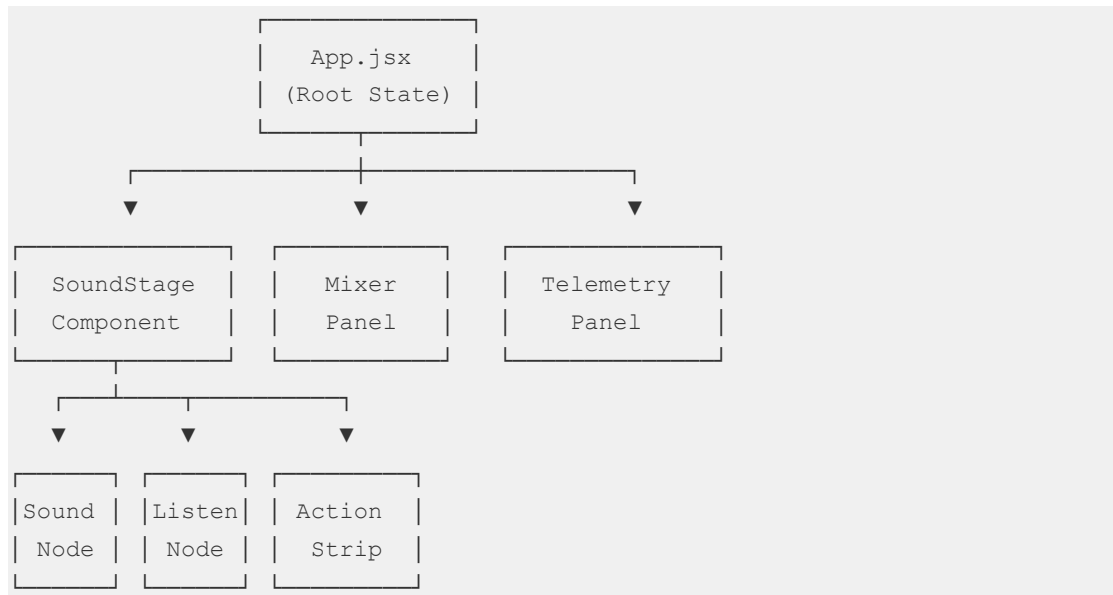
The application uses React's `useState` and `useEffect` hooks for state management. The central sources array in `App.jsx` acts as the single source of truth. When any source property changes, the `useEffect` hooks propagate updates to both the Web Audio engine and browser `localStorage` for persistence.



8. COMPONENT ARCHITECTURE

The application is organized into a modular component hierarchy following React best practices. Each component has a clearly defined responsibility and communicates with its parent through props and callback functions.

React Component Hierarchy



App.jsx — Root Component & State Manager

Purpose: Central orchestrator managing all application state and business logic.

Responsibilities:

- Maintains the `master sources[]` array containing all audio node configurations.
- Performs spatial mathematics (distance, panning, attenuation) via `updateSpatialMath()`.
- Synchronizes React state with the Web Audio engine on every user interaction.
- Handles `localStorage` persistence and JSON export/import operations.
- Maps Y-axis positions to oscillator frequencies using exponential interpolation.

SoundStage.jsx — Interactive 2D Canvas

Purpose: Provides the visual 2D grid where audio sources can be positioned.

Responsibilities:

- Renders a grid-based stage with a central listener node and draggable sound source nodes.
- Implements pointer event-based drag-and-drop (supporting both mouse and touch).
- Uses HTML5 Canvas overlays to draw dashed distance vector lines from each source to the listener.

- Renders glowing proximity collision bridges between overlapping sources.
- Supports grid snapping mode for precise node positioning.

Mixer.jsx — Channel Mixer Panel

Purpose: Provides per-channel controls for audio parameter adjustments.

- Displays a card for each audio source with volume slider, pan slider, and control buttons.
- Renders live waveform visualizations using `requestAnimationFrame` and `AnalyserNode` data.
- Provides play/pause, mute/unmute, and delete controls per channel.
- Bidirectionally synchronizes pan slider values with node positions on the stage.

TelemetryPanel.jsx — Diagnostics Dashboard

Purpose: Displays real-time system diagnostics and per-channel audio parameters.

- Shows `AudioContext` state (running/suspended), sample rate, active channel count, and collision count.
- Displays base latency and output latency metrics.
- Renders a tabular readout of per-channel coordinates, distance, stereo pan, and attenuation factor.
- Provides attenuation model selection (Linear or Exponential).
- Includes Export JSON and Clear Local Cache utility buttons.

ActionStrip.jsx — Toolbar Controls

- Add Oscillator Node, Upload Audio File, Mute Master buttons.
- Grid Snapping toggle control.
- Export JSON, Import JSON, Reset Default, and Clear Stage operations.

Audio Engine — Web Audio API Abstraction Layer

Purpose: Encapsulates all Web Audio API operations behind a clean interface.

- Manages `AudioContext` lifecycle (initialization, resumption, suspension).
- Creates and connects audio node chains (Source → Panner → Gain → Destination).
- Handles oscillator creation and audio file decoding.
- Provides waveform analysis data via `AnalyserNode` for the mixer visualizer.
- Manages proximity effects (tremolo, lowpass filter) for colliding nodes.

9. CORE FEATURES

9.1 Interactive 2D Sound Stage

The sound stage is a grid-based 2D canvas representing the listening environment. It features a central fixed listener node and supports multiple draggable audio sources. The stage includes a rotating radar sweep animation, grid lines, and dynamic distance vector overlays drawn on an HTML5 Canvas layer.

9.2 Draggable Audio Sources

Audio sources are represented as draggable nodes on the stage. Users can click and drag nodes using pointer events, which seamlessly support both mouse and touchscreen interactions. When a node is dragged, the system continuously recalculates its spatial audio parameters (pan, attenuation, frequency) and updates both the visual display and audio output in real time.

9.3 Real-Time Stereo Panning

The horizontal position (X-axis) of each audio source directly controls its stereo pan value. Sources positioned to the left of the listener produce sound primarily in the left speaker, while sources on the right produce sound in the right speaker. The pan value is linearly mapped from the X-coordinate range $[0, 100]$ to the Web Audio API's StereoPannerNode range $[-1.0, +1.0]$.

9.4 Distance-Based Attenuation

Volume attenuation simulates the natural phenomenon of sound growing quieter as the source moves further from the listener. The system supports two attenuation models: Linear $(1 - d/d_{\text{max}})$ and Exponential $(1 - d/d_{\text{max}})^2$. Users can switch between models via the telemetry panel.

9.5 Multi-Channel Audio Processing

The system supports an arbitrary number of simultaneously active audio channels. Each channel maintains its own independent audio processing chain (source → panner → gain → destination) and can be individually controlled through the mixer panel.

9.6 Oscillator-Based Audio Sources

Users can create synthesizer-based sound sources that generate tones using the Web Audio API's OscillatorNode. Four waveform types are supported: Sine, Square, Sawtooth, and Triangle. The Y-axis position can be mapped to oscillator frequency using an exponential interpolation between 80 Hz and 2000 Hz.

9.7 Audio File Upload Support

The system accepts user-uploaded audio files (.mp3, .wav) which are decoded using the AudioContext's decodeAudioData method and played back as spatial audio sources with full panning and attenuation support.

9.8 Proximity Interaction Detection

When two audio sources are positioned within a configurable proximity threshold, the system detects the collision and applies visual effects (glowing bridge lines on the canvas) and audio effects (tremolo modulation, lowpass filter sweeps) proportional to the overlap intensity.

9.9 Mixer Controls

The mixer provides per-channel volume sliders (0-100%), pan sliders (-1.0 to +1.0), play/pause toggle, mute/unmute toggle, and channel deletion. Live waveform visualizers render the time-domain audio data for each channel using `requestAnimationFrame`.

9.10 Telemetry Dashboard

The telemetry panel provides real-time diagnostics including `AudioContext` state, hardware sample rate, active channel count, collision count, base latency, and output latency. A tabular readout displays per-channel coordinates, distance, pan value, and attenuation factor.

9.11 JSON Export/Import

Users can export the entire stage configuration (source positions, types, volumes, settings) as a JSON file. This file can later be imported to restore the exact stage layout, enabling scene sharing and backup functionality.

9.12 Local Storage Persistence

The application automatically saves the current state to browser `localStorage` whenever sources or settings change. On page reload, the saved state is restored, providing session continuity without requiring a backend server.

10. SPATIAL AUDIO MATHEMATICS

The spatial audio simulation relies on several mathematical concepts to calculate audio parameters from source positions. These calculations run on every frame during drag operations to provide real-time audio updates.

10.1 Coordinate System

The sound stage uses a percentage-based coordinate system where both X and Y range from 0 to 100. The listener is fixed at position (50, 50), representing the center of the stage. This percentage-based system ensures consistent behavior regardless of the browser window size or screen resolution.

10.2 Euclidean Distance Formula

The distance between an audio source at position (x, y) and the listener at position (50, 50) is calculated using the Euclidean distance formula:

$$\text{Distance} = \sqrt{(x - 50)^2 + (y - 50)^2}$$

This formula calculates the straight-line distance between two points in 2D space. The maximum possible distance occurs when a source is at a corner of the stage (e.g., position 0,0), giving a maximum distance of $\sqrt{50^2 + 50^2} \approx 70.71$ units.

10.3 Stereo Panning Logic

Stereo panning is determined by the horizontal offset of the source from the listener. The X-axis displacement is normalized to the range [-1.0, +1.0]:

$$\text{Pan} = \text{clamp}((x - 50) / 50, -1.0, +1.0)$$

- $x = 0 \rightarrow \text{Pan} = -1.0$ (Full Left)
- $x = 50 \rightarrow \text{Pan} = 0.0$ (Center)
- $x = 100 \rightarrow \text{Pan} = +1.0$ (Full Right)

10.4 Volume Attenuation Logic

Volume attenuation simulates the inverse relationship between distance and perceived loudness. Two mathematical models are supported:

$$\text{Linear Model: } \text{Attenuation} = \max(0, 1 - \text{distance} / 70.71)$$

$$\text{Exponential Model: } \text{Attenuation} = \max(0, (1 - \text{distance} / 70.71)^2)$$

The linear model provides a uniform rate of volume decrease. The exponential model produces a steeper initial drop-off with a more gradual tail, which more closely approximates real-world sound propagation where the inverse-square law governs intensity.

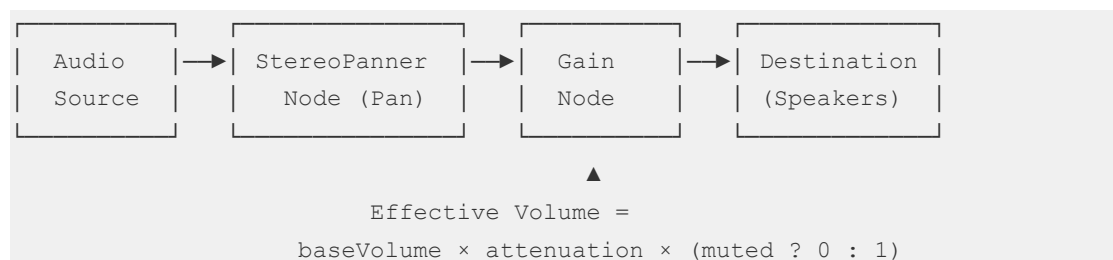
10.5 Spatial Position Mapping (Y-Axis to Frequency)

For oscillator sources, the Y-axis position is optionally mapped to frequency using exponential interpolation:

$$\text{Frequency} = \text{FREQ_MIN} \times (\text{FREQ_MAX} / \text{FREQ_MIN})^{(1 - y/100)}$$

Where: FREQ_MIN = 80 Hz, FREQ_MAX = 2000 Hz. Top of stage ($y=0$) $\rightarrow$ 2000 Hz (high pitch); Bottom of stage ($y=100$) $\rightarrow$ 80 Hz (low pitch). Exponential interpolation is used because human pitch perception follows a logarithmic scale.

10.6 Audio Signal Flow



11. IMPLEMENTATION DETAILS

11.1 Drag-and-Drop System

The drag-and-drop system uses Pointer Events (pointerdown, pointermove, pointerup) rather than traditional mouse events. Pointer Events provide unified handling for mouse, touch, and pen inputs. The event listeners are attached to the window object during drag operations to ensure smooth tracking even when the cursor moves outside the stage container boundaries.

During a drag operation, the system: (1) calculates percentage coordinates from pixel positions, (2) applies optional grid snapping (rounding to 5% increments), (3) clamps values to the [0, 100] range, and (4) triggers the spatial math recalculation and audio engine update.

11.2 Audio Context Initialization

Modern browsers enforce an autoplay policy that requires user interaction before allowing audio playback. The AudioContext starts in a 'suspended' state and must be explicitly resumed after a user gesture. The application handles this by initializing and resuming the AudioContext on the first user interaction (button click or node drag).

11.3 Gain Node Processing

Each audio source has a dedicated GainNode that controls its output volume. The effective volume is calculated as:

```
effectiveVolume = muted ? 0 : (baseVolume × attenuation)
```

The GainNode's gain.value property is updated directly (bypassing React re-renders) during drag operations for optimal performance.

11.4 Collision Detection Logic

Pairwise collision detection runs after every source position update. For each pair of sources, the system calculates their Euclidean distance and compares it against the PROXIMITY_THRESHOLD (12 units). When overlapping, a collision intensity value is computed as $(1 - \text{distance}/\text{threshold})$, ranging from 0 (just touching) to 1 (perfectly overlapping). This intensity drives both visual effects and audio modulation parameters.

11.5 Browser Storage Handling

State persistence uses a useEffect hook that serializes the sources array and settings to localStorage as a JSON string whenever they change. On mount, a corresponding useEffect reads from localStorage and restores the saved state. Audio sources are initialized in a paused state to comply with browser autoplay policies.

11.6 Performance Considerations

To minimize rendering lag during continuous drag operations, the application uses a 'fast-path' approach: audio engine parameters (gain, pan, frequency) are updated

directly via the Web Audio API without waiting for React re-renders. React state is still updated for visual synchronization, but the audio output responds instantly to position changes.

12. USER WORKFLOW

The following describes the step-by-step workflow that a user follows when interacting with the application:

Step	Action	System Response
1	User launches application	React renders the UI. Default oscillator sources are loaded from localStorage or defaults. AudioContext is created in suspended state.
2	Audio system initializes	On first user interaction, AudioContext.resume() is called. The engine status indicator changes from 'SUSPENDED' (amber) to 'RUNNING' (green).
3	User creates a sound source	A new oscillator or file node is created, registered in the audio engine, and rendered on the stage at a random or clicked position.
4	User drags source on stage	Pointer events track the cursor. Percentage coordinates are computed and passed to the spatial math calculator.
5	Distance recalculates	Euclidean distance from source to listener is computed. Canvas overlay redraws the dashed vector line with the distance label.
6	Panning updates	Horizontal offset is normalized to [-1, +1] and applied to the StereoPannerNode. Audio shifts between speakers accordingly.
7	Telemetry updates	Telemetry panel refreshes with new distance, pan, and attenuation values for the moved source.
8	User exports configuration	Current stage layout is serialized to JSON and downloaded as a file via a dynamically created anchor element.
9	State persists	useEffect hook detects state change and writes the serialized sources array to localStorage automatically.

13. SCREENSHOTS

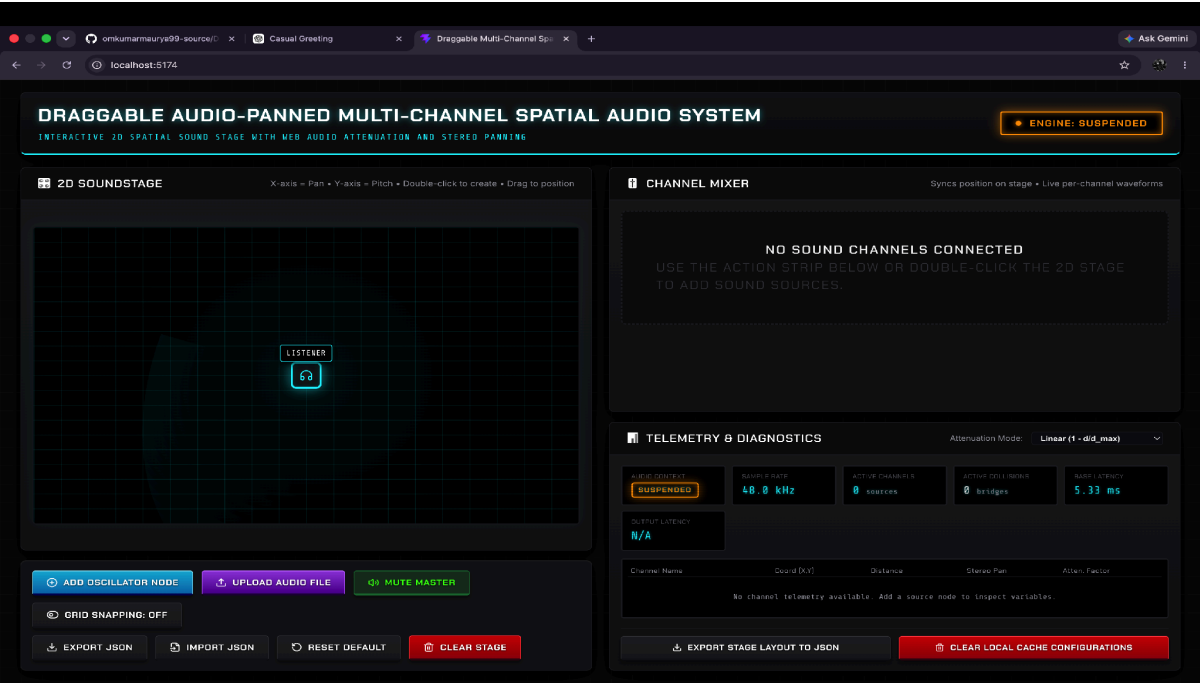


Figure 1: Main Dashboard — Complete application interface showing the sound stage, mixer panel, and telemetry dashboard in the dark-themed studio layout.

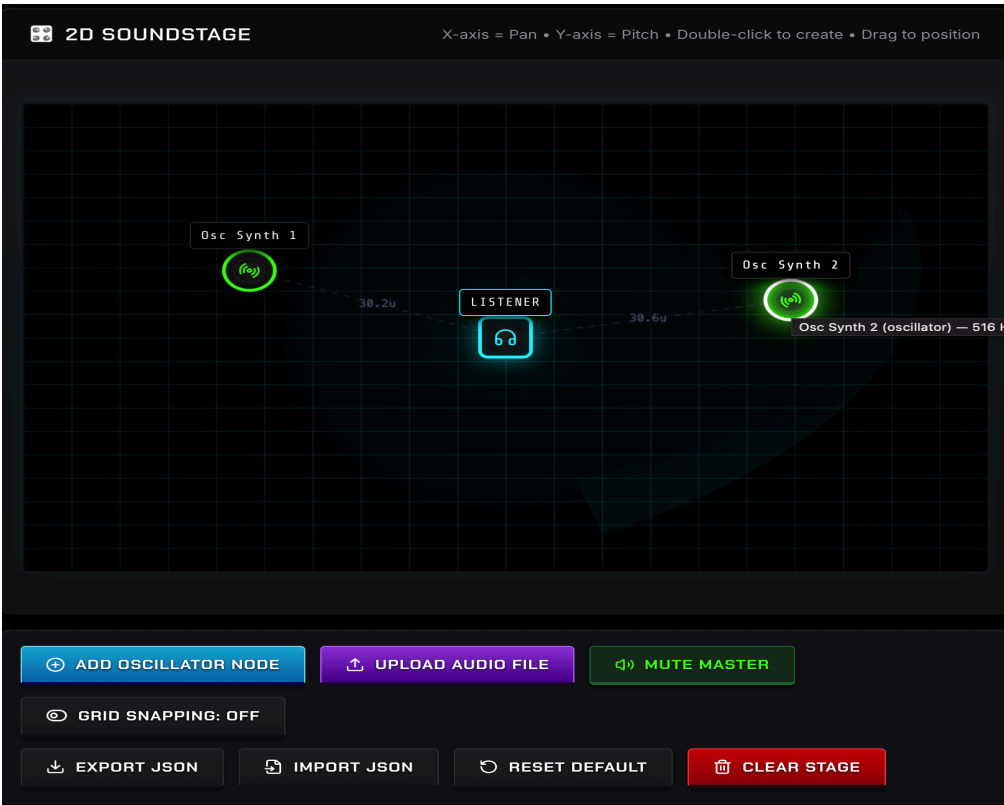


Figure 2: Interactive Sound Stage — 2D grid with the central listener node, draggable audio source nodes, radar sweep animation, and dashed distance vector lines.

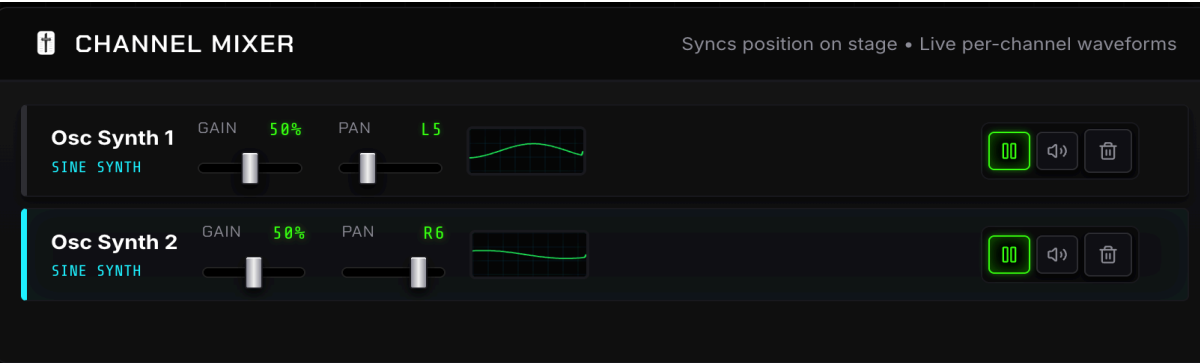


Figure 3: Channel Mixer — Per-channel volume and pan sliders, live waveform visualizers, play/pause, mute, and delete controls.

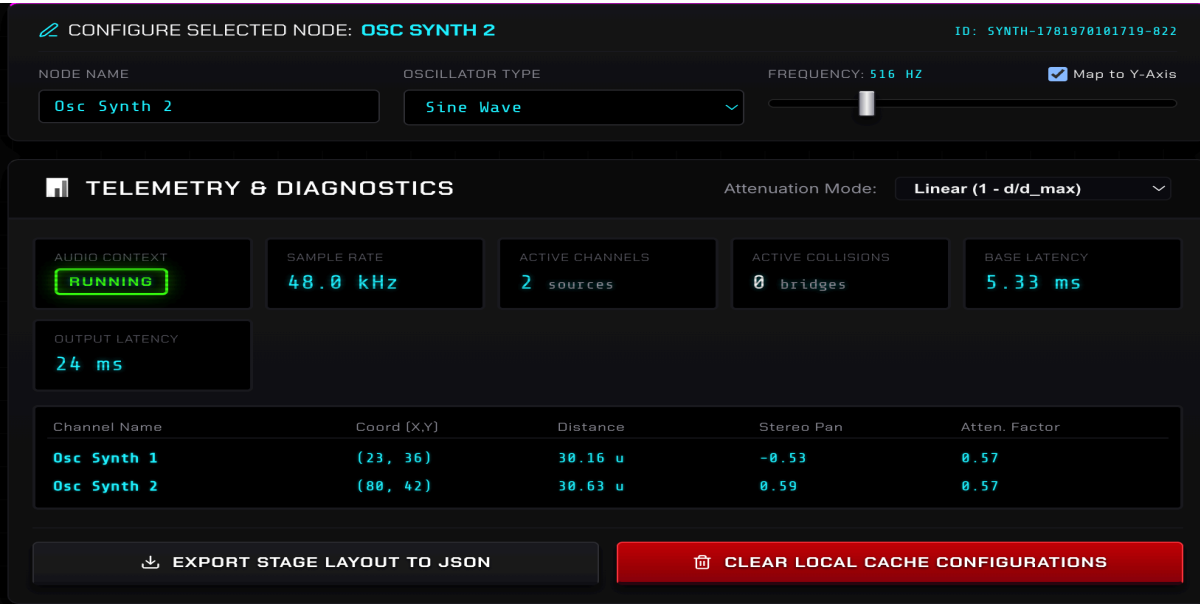


Figure 4: Telemetry & Diagnostics Panel — AudioContext status, sample rate, latency metrics, per-channel coordinate readouts, and attenuation model selector.

14. TESTING & VERIFICATION

14.1 Functional Testing

All core application functions were verified through manual testing procedures. Each feature was tested in isolation and in combination with other features to ensure correct behavior under various conditions.

Test Case	Status	Description
Add Oscillator Source	✓ Passed	New oscillator nodes are created with correct default parameters and rendered on the stage.
Upload Audio File	✓ Passed	Audio files are successfully decoded and playback begins with spatial processing applied.

Test Case	Status	Description
Drag Node on Stage	✓ Passed	Nodes follow cursor accurately. Audio parameters update in real time without perceptible lag.
Stereo Panning	✓ Passed	Moving a node left/right produces corresponding shift in audio output between speakers.
Distance Attenuation	✓ Passed	Moving a node away from the listener reduces volume. Both linear and exponential models function correctly.
Mixer Volume Control	✓ Passed	Volume slider adjustments are reflected in audio output immediately.
Mute/Unmute	✓ Passed	Muting a channel silences its output; unmuting restores previous volume level.

14.2 UI Testing

The user interface was tested across multiple screen sizes and browsers to verify responsive layout behavior, element visibility, and interaction responsiveness. All controls remain accessible and functional across Chrome, Firefox, and Safari on desktop platforms.

14.3 Audio Testing

Audio behavior was verified by listening to the output while performing drag operations. The stereo panning effect was confirmed using headphones, verifying that leftward movement produces left-channel dominance and rightward movement produces right-channel dominance. Attenuation was verified by moving sources to the corners of the stage and confirming near-silence at maximum distance.

14.4 Persistence Testing

State persistence was verified by: (1) adding multiple sources with custom positions and settings, (2) reloading the browser page, and (3) confirming that all sources are restored in their correct positions with correct parameters. JSON export/import was tested by exporting a configuration, clearing the stage, and importing the same file to verify exact restoration.

14.5 Deployment Testing

The application was deployed to Vercel and verified to be accessible at the public URL. All features were re-tested on the deployed version to confirm that the production build functions identically to the development environment.

15. CHALLENGES FACED

15.1 Web Audio API Integration

Integrating the Web Audio API with React's rendering cycle presented significant challenges. The `AudioContext` must be initialized after a user gesture due to browser autoplay policies, requiring careful coordination between React lifecycle events and audio system initialization. Additionally, directly manipulating audio node parameters during rapid drag events required bypassing React's state update cycle to avoid rendering lag.

15.2 Real-Time Audio Synchronization

Maintaining synchronization between the visual representation (React state and DOM updates) and the audio output (Web Audio API parameter changes) during continuous drag operations required implementing a dual-update strategy: audio parameters are updated directly via the Web Audio API for immediate audible feedback, while React state is updated concurrently for visual consistency.

15.3 Spatial Audio Calculations

Implementing accurate spatial audio mathematics required careful consideration of coordinate system mapping, edge cases (sources at exact center, sources at stage boundaries), and the selection of appropriate attenuation models. The exponential frequency mapping for oscillator sources required understanding of logarithmic perception scales in human hearing.

15.4 State Persistence

Serializing and deserializing the complete application state, including audio source configurations, settings, and positions, required handling edge cases such as missing fields in older saved states, corrupted `localStorage` data, and audio file references that cannot be serialized (requiring users to re-upload files after import).

16. LEARNING OUTCOMES

Through the development of this project, the following key competencies were developed:

Skill Area	What Was Learned
React Development	Proficiency in component-based architecture, props/callback communication, hooks (<code>useState</code> , <code>useEffect</code> , <code>useRef</code>), and lifecycle management.
Component Architecture	Designing modular, reusable components with clearly defined responsibilities and unidirectional data flow.

Skill Area	What Was Learned
State Management	Centralized state management using React hooks, lifting state up, and synchronizing state with external systems (Web Audio API, localStorage).
Web Audio API	Deep understanding of AudioContext, OscillatorNode, GainNode, StereoPannerNode, AnalyserNode, and audio routing graphs.
Spatial Audio Systems	Mathematical foundations of spatial audio including Euclidean distance, coordinate mapping, stereo panning, and distance attenuation models.
Frontend Engineering	Modern build tooling (Vite), deployment pipelines (Vercel), responsive design, canvas rendering, and performance optimization.
UI/UX Design	Creating intuitive, visually appealing interfaces with real-time feedback, drag interactions, and professional dark-themed styling.

17. FUTURE IMPROVEMENTS

The following enhancements are planned for future iterations of the project:

8. 3D Spatial Audio: Upgrade from 2D stereo panning to full 3D spatialization using the PannerNode, introducing Z-axis depth and elevation controls for a more immersive experience.
9. Reverb and Environment Effects: Integrate ConvolverNode to simulate different acoustic environments (concert hall, small room, cathedral, open field) with adjustable wet/dry mix.
10. Mobile Touch Optimization: Enhance multi-touch support for simultaneous dragging of multiple audio sources on mobile and tablet devices.
11. Audio Visualizers: Add frequency spectrum analyzers and waveform oscilloscopes as larger, dedicated visualization panels with configurable FFT sizes.
12. Collaborative Mode: Implement WebSocket-based real-time collaboration allowing multiple users to interact with the same sound stage simultaneously.
13. MIDI Controller Support: Enable external MIDI hardware controllers to manipulate mixer parameters, providing a tactile interface for audio control.
14. Recording and Playback: Integrate MediaRecorder API to capture the spatial audio output as a downloadable audio file.

18. CONCLUSION

The Draggable Audio-Panned Multi-Channel Spatial Audio System successfully achieves all its stated objectives. The application provides a fully functional, interactive 2D spatial audio simulator that operates entirely within a modern web browser without requiring any server-side infrastructure or plugin installations.

The project demonstrates the practical application of multiple computer science concepts: component-based software architecture through React, real-time audio processing through the Web Audio API, coordinate geometry through Euclidean distance calculations, and data persistence through browser storage mechanisms. The telemetry dashboard provides transparent insight into the mathematical operations driving the audio processing, reinforcing the educational value of the system.

The application has been successfully deployed to Vercel and is publicly accessible, demonstrating a complete software development lifecycle from design through implementation to deployment. The codebase follows modern development practices including modular component design, clean separation of concerns, and comprehensive state management.

As a first-year B.Tech CSE project, this system represents a meaningful exploration of frontend engineering, audio processing, and interactive application design. The skills and knowledge gained through its development provide a strong foundation for future work in web application development and multimedia systems.

19. REFERENCES

15. React Documentation. (2026). React - A JavaScript library for building user interfaces. Retrieved from <https://react.dev>
16. Vite Documentation. (2026). Vite - Next Generation Frontend Tooling. Retrieved from <https://vitejs.dev>
17. Mozilla Developer Network. (2026). Web Audio API. Retrieved from https://developer.mozilla.org/en-US/docs/Web/API/Web_Audio_API
18. Mozilla Developer Network. (2026). JavaScript Reference. Retrieved from <https://developer.mozilla.org/en-US/docs/Web/JavaScript>
19. Mozilla Developer Network. (2026). Pointer Events. Retrieved from https://developer.mozilla.org/en-US/docs/Web/API/Pointer_events
20. Mozilla Developer Network. (2026). Window.localStorage. Retrieved from <https://developer.mozilla.org/en-US/docs/Web/API/Window/localStorage>

Project Links

Resource	URL
GitHub Repository	https://github.com/omkumarmaurya99-source/Draggable-Audio-Panned-Multi-Channel-Playground.git
Live Demo	https://draggable-audio-panned-multi-CHANNE-five.vercel.app/

End of Document

LISA Documentation — Om Kumar Maurya — B.Tech CSE (First Year) — 2026